Analyse des impacts énergétiques du compilateur `decac` et propositions d'amélioration

Projet GL — Équipe `gl53`

23 janvier 2026

Table des matières

1 Analyse des impacts énergétiques et propositions **2**
1.1 Méthodologie d'évaluation et métriques . . . 2
1.2 Efficience du code produit : impact de nos choix de génération . . . 2
1.2.1 Allocation de registres : réduire les accès mémoire . . . 2
1.2.2 Gestion du spill (saturation des registres) . . . 3
1.2.3 Évaluation paresseuse des booléens (AND/OR) . . . 3
1.2.4 Constant folding : tentative puis abandon pour robustesse . . . 3
1.3 Robustesse et "sobriété par la fiabilité" : réduire les relances et le coût global . . . 4
1.3.1 Suivi de la taille de pile et dimensionnement de `TSTO` . . . 4
1.3.2 Gestion systématique des erreurs d'exécution . . . 4
1.3.3 Factorisation des motifs de génération de code . . . 4
1.4 Efficience du procédé de fabrication : compilation et validation . . . 4
1.4.1 Stratégie de validation : limiter l'exécution inutile . . . 5
1.4.2 Scripts et exécution : réduire le coût système . . . 5
1.5 Partie objet : choix de sûreté vs optimisations . . . 5
1.5.1 Construction explicite des vtables : choix clair et prédictible . . . 5
1.5.2 Prologue/épilogue conservateur dans les méthodes . . . 5
1.6 Mesures `ima -s` : comparaison avec le compilateur du professeur . . . 5
1.7 Mesures côté Java : compilation en série vs `-P` (parallèle) . . . 6

2 Propositions d'optimisation (pistes futures) **7**
2.1 Optimisations côté code produit (IMA) . . . 7
2.1.1 Prologue/épilogue de méthodes plus sélectif . . . 7
2.1.2 Constant folding : réintroduction progressive et sûre . . . 7
2.1.3 Réduction des `LOAD` inutiles : instructions IMA à opérande mémoire . . . 7
2.2 Optimisations côté procédé de fabrication (Java/Maven/Tests) . . . 8
2.2.1 Exécution parallèle des tests : à utiliser avec discernement . . . 8

1 Analyse des impacts énergétiques et propositions

Dans un contexte de prise de conscience écologique globale, le développement logiciel se doit d'intégrer la dimension de l'efficience énergétique. Pour un compilateur comme **decac**, cette responsabilité est double :

- **Efficience du code produit :** réduire l'énergie consommée à l'exécution par les programmes générés.
- **Efficience du procédé de fabrication :** limiter le coût énergétique de la compilation et surtout des campagnes de tests répétées pendant le projet.

1.1 Méthodologie d'évaluation et métriques

Nous avons utilisé deux types d'indicateurs, adaptés au cadre du projet :

- **Code produit (exécution) :** le simulateur `ima` fournit un **nombre de cycles** (option `-s`), que l'on considère comme une approximation raisonnable de la consommation énergétique d'un programme sur la machine virtuelle.
- **Procédé de fabrication (compilation + tests) :** la commande `/usr/bin/time` (temps *user+sys*) donne une estimation indirecte du coût CPU (donc énergie) de la compilation et de l'exécution des scripts de validation.

1.2 Efficience du code produit : impact de nos choix de génération

Notre objectif principal a été de réduire les accès mémoire (pile/tas), généralement plus coûteux que les opérations en registres. Dans notre compilation **Deca** → **IMA**, cela se traduit par :

- privilégier les calculs en registres pour les expressions ;
- utiliser la pile uniquement en cas de saturation (spill) ou lors des conventions d'appel (notamment en partie objet).

1.2.1 Allocation de registres : réduire les accès mémoire

Nous avons implémenté une politique simple (allocateur linéaire) qui cherche à garder les valeurs intermédiaires dans les registres $R_2 \dots R_{max}$ (où R_{max} dépend de `decac -r`).

Idée énergétique. Une stratégie "tout-pile" conduirait à de nombreux `PUSH`/`POP` (écritures/-lectures mémoire), augmentant le nombre de cycles. À l'inverse, une stratégie "registre d'abord" réduit le trafic mémoire et donc les cycles sur `ima`.

Exemple simple (comparatif conceptuel). Pour calculer `a + b`, une stratégie pile ajoute des accès mémoire supplémentaires (sauvegarde/restauration), alors que notre stratégie vise à maintenir les opérandes en registres puis effectuer directement l'opération.

Approche "pile" (concept)	Approche "registres" (concept)
sauvegardes via `PUSH/POP`	calcul direct en R_i
plusieurs `LOAD/STORE`	moins de `LOAD/STORE`

Table 1 – Objectif : diminuer le trafic mémoire → moins de cycles → énergie réduite.

1.2.2 Gestion du spill (saturation des registres)

Quand une expression devient trop complexe (ex : `(x+4)+(2+x)`) et que les registres disponibles sont saturés (ex : `decac -r 4`), le compilateur doit utiliser la pile. Dans notre projet, ce cas est **centralisé** dans `AbstractBinaryExpr` (c'est le seul nœud « parent » qui teste `hasNext()`), ce qui évite de dupliquer la logique de spill dans plusieurs classes.

Exemple issu d'un test (spill effectif).

```
LOAD 5(GB), R3
PUSH R3
LOAD 6(GB), R3
PUSH R3
LOAD -1(SP), R3
LOAD 0(SP), R0
ADD R0, R3
SUBSP #2
ADD R3, R2
```

Approche suivie. Le spill se fait localement au niveau des expressions binaires :

- lorsque aucun registre libre n'est disponible, on sauvegarde temporairement une valeur intermédiaire sur la pile (`PUSH`) ;
- on calcule l'autre opérande en registre ;
- on récupère ensuite les valeurs depuis la pile via des `LOAD` (`-1(SP)` et `0(SP)`) ;
- on effectue l'opération binaire (ici `ADD`) en utilisant `R0` comme registre temporaire ;
- on nettoie la pile en une seule instruction (`SUBSP #2`) pour éviter des `POP` supplémentaires.

Impact énergétique attendu. Ce schéma limite les accès pile au strict nécessaire lors d'un débordement de registres, ce qui réduit le nombre de cycles IMA sur des expressions imbriquées (arithmétique, affectations, etc.).

1.2.3 Évaluation paresseuse des booléens (AND/OR)

Pour et `||`, nous utilisons un schéma de branchement (short-circuit) :

- dans `a && b`, si `a` est faux, `b` n'est pas évalué ;
- dans `a || b`, si `a` est vrai, `b` n'est pas évalué.

Intérêt énergétique. Sur des conditions fréquentes en boucles, éviter des évaluations inutiles réduit directement les cycles (donc énergie) et diminue aussi la pression sur registres/pile.

1.2.4 Constant folding : tentative puis abandon pour robustesse

Nous avions implémenté une optimisation de **repli de constantes** (ex. `1+2` calculé à la compilation). Cette optimisation peut réduire le nombre d'instructions générées, donc les cycles à l'exécution.

Pourquoi l'avoir annulée ? Nous avons privilégié la **robustesse** :

- sur des expressions complexes, le constant folding demandait une gestion très fine des cas limites (débordements, cohérence des messages d'erreurs, comportements attendus avec `BOV`, etc.) ;
- cette complexité supplémentaire augmentait le risque de bugs et rendait la validation plus coûteuse.

Dans le cadre du projet, nous avons estimé que le gain énergétique potentiel ne justifiait pas le risque sur la correction et la qualité des diagnostics.

1.3 Robustesse et "sobriété par la fiabilité" : réduire les relances et le coût global

Au-delà des optimisations du code produit, une partie importante de l'impact énergétique vient du **cycle de développement** : un compilateur plus stable engendre moins de débogage et moins de campagnes de tests répétées.

1.3.1 Suivi de la taille de pile et dimensionnement de `TSTO`

Nous avons instrumenté `DecacCompiler.addInstruction()` pour maintenir un suivi simple de la pile (`currentStackSize` / `maxStackSize`) en mettant à jour ces compteurs lors des `PUSH`/`POP` et en tenant compte des variations de `SUBSP`. Cette information est ensuite utilisée pour dimensionner `TSTO` en début de programme (`Program.codeGenProgram`) de manière plus réaliste.

Impact énergétique.
- un `TSTO` mieux dimensionné limite les erreurs d'exécution et donc les relances inutiles de tests ;
- éviter des débordements de pile en exécution réduit aussi les exécutions interrompues (cycles perdus).

1.3.2 Gestion systématique des erreurs d'exécution

Notre compilateur génère des labels et routines d'erreurs standard (`stack_overflow_error`, `arithmetic_overflow_error`, `division_zero_error`, `null_pointer_error`, etc.). De plus, lorsque `-n` n'est pas activé, nous insérons des vérifications (`CMP null` puis `BEQ`) avant les déréférencements d'objets/champs et certains chemins sensibles.

Impact énergétique.
- léger surcoût en instructions sur les chemins vérifiés ;
- gain important en robustesse : une exécution qui échoue proprement évite des comportements indéfinis et accélère la mise au point.

1.3.3 Factorisation des motifs de génération de code

Nous avons cherché à **factoriser** les motifs communs au lieu de dupliquer du code :
- `AbstractBinaryExpr` factorise la stratégie de calcul pour les opérations binaires arithmétiques (hors booléens), et centralise le spill.
- `DecacCompiler` fournit une interface unique d'émission d'instructions et centralise les labels d'erreurs.

Impact énergétique. Moins de duplication → moins de divergences entre cas → moins de bugs → moins d'itérations de validation (compilations + tests), donc un coût énergétique plus faible côté "fabrication".

1.4 Efficience du procédé de fabrication : compilation et validation

Pendant le projet GL, la consommation énergétique est principalement due aux **compilations répétées** et aux **campagnes de tests**.

1.4.1 Stratégie de validation : limiter l'exécution inutile

Nos choix ont visé à réduire le nombre d'exécutions inutiles tout en conservant une bonne couverture :

- exécuter localement un sous-ensemble ciblé de tests après une modification (tests concernés uniquement) ;
- réserver les campagnes complètes (tous les répertoires, toutes les options) aux étapes de stabilisation.

1.4.2 Scripts et exécution : réduire le coût système

Les tests système (scripts shell lançant `decac` puis `ima`) sont plus coûteux car ils impliquent des processus externes et des entrées/sorties. Nous avons donc :

- regroupé les tests par catégorie (contextuel, codegen, invalid) pour lancer uniquement ce qui est nécessaire ;
- évité les boucles "trop générales" lorsqu'un échec est détecté tôt (fail-fast), afin d'économiser des exécutions longues.

1.5 Partie objet : choix de sûreté vs optimisations

1.5.1 Construction explicite des vtables : choix clair et prédictible

Pour la partie objet, nous avons construit les vtables en recopiant la vtable du parent puis en remplaçant les entrées en cas d'override. Concrètement, nous avons :

- initialisé la vtable racine avec `code.Object.equals` ;
- parcouru récursivement l'héritage pour récupérer la liste de labels ;
- remplacé une entrée existante si une méthode est redéfinie (override), sinon ajouté en fin.

Impact énergétique. Ce mécanisme rend le dispatch dynamique **direct** (chargement vtable puis `BSR` à l'index), donc peu coûteux en cycles et **stable** (moins d'effets de bord → moins de bugs).

1.5.2 Prologue/épilogue conservateur dans les méthodes

Dans la génération de code des méthodes, nous sauvegardons/restaurons de manière **conservatrice** les registres (`PUSH R2..Rmax` puis `POP` en sortie), avec un `TSTO/BOV` au début.

Justification énergétique (paradoxale mais assumée). Sauvegarder uniquement les registres réellement vivants serait plus optimal en cycles, mais :

- cela nécessite une analyse plus fine (liveness) et une gestion plus complexe ;
- le risque de bugs est plus élevé, ce qui augmente le coût global via davantage de débogage et davantage d'exécutions de tests.

Nous avons donc privilégié la correction et la stabilité : un léger surcoût à l'exécution d'une méthode, mais un gain sur le coût énergétique du développement (moins de retours en arrière).

1.6 Mesures `ima -s` : comparaison avec le compilateur du professeur

Pour objectiver l'efficience énergétique du **code produit**, nous avons comparé nos résultats avec ceux du compilateur de référence (professeur) en utilisant `ima -s` sur trois programmes représentatifs de notre extension (TRIGO) et des tests de performance.

Résultats (nos mesures `ima -s`).

- **Syracuse (S)** : **77** instructions, **1764** cycles
- **ln2 (L0)** : **135** instructions, **17690** cycles
- **ln2_fct (L1)** : **192** instructions, **21773** cycles // mais avant le sauvegarde des registres au debut d'une methode, on avait :**170** instructions, **20189** cycles

Remarque sur `ln2_fct`. Nous observons **22 instructions supplémentaires** sur `ln2_fct`, expliquées par notre choix **conservateur** de sauvegarde/restauration des registres au prologue/épilogue avant l'appel de méthode. Ce surcoût local est assumé : il améliore la robustesse (moins de risques d'écrasement de registres), ce qui réduit les cycles perdus en débogage et en relances de campagnes de tests au niveau du projet.

Comparaison globale (tableau de scores). Le tableau de synthèse ci-dessous compare le score agrégé `Score = L0 + L1 + 10*S`.

Équipe	S (Syracuse)	L0 (ln2)	L1 (ln2_fct)	Score
VieuxCompDeProf (TRIGO)	1340	15194	18102	46696
Notre compilateur (TRIGO)	**1764**	**17690**	**21773**	**57127**

Table 2 – Comparaison `ima -s` (cycles) et score agrégé.

Interprétation. Notre score est plus élevé que celui du compilateur de référence sur ces trois programmes, avec un écart notable sur `ln2` et `ln2_fct`. Cet écart est cohérent avec nos choix d'implémentation :

- une génération de code **robuste** (vérifications, gestion conservatrice des registres en méthodes) ;
- une priorisation de la **stabilité et des diagnostics** pendant le projet, parfois au détriment d'une optimisation agressive.

Ces résultats fournissent une base mesurée pour identifier, si nécessaire, des pistes d'amélioration ciblées (prologue/épilogue plus fin, réduction de certaines séquences mémoire), tout en conservant les garanties de correction.

1.7 Mesures côté Java : compilation en série vs `-P` (parallèle)

Pour estimer l'impact énergétique du **procédé de fabrication** (exécution du compilateur Java), nous avons mesuré le coût CPU avec `/usr/bin/time` lors de la compilation d'un lot de fichiers `valid`.

Commande	real	user	sys
`decac valid/*.deca`	0.820 s	1.747 s	0.215 s
`decac -P valid/*.deca`	0.727 s	2.450 s	0.374 s

Table 3 – Temps mesurés (`/usr/bin/time`) sur un lot de fichiers `valid`.

Mesures.

Interprétation énergétique. Le temps mur (`real`) diminue avec `-P` (0.820 s → 0.727 s), ce qui améliore la réactivité. En revanche, le **temps CPU total** augmente, ce qui est un bon proxy d'une hausse de consommation énergétique côté compilation :

$$CPU = user + sys.$$

Ici, on passe de $1.747 + 0.215 = 1.962$ s à $2.450 + 0.374 = 2.824$ s, soit environ **+44%** de CPU.

Conclusion.
- **Si l'objectif est la vitesse (temps mur)** : `-P` est intéressant.
- **Si l'objectif est la sobriété énergétique CPU** : la compilation en série est potentiellement plus efficiente, car elle consomme moins de temps CPU total.

2 Propositions d'optimisation (pistes futures)

Cette section liste des optimisations réalistes que nous pourrions mettre en œuvre pour réduire l'impact énergétique, à la fois sur le **code produit** (cycles IMA) et sur le **procédé de fabrication** (compilation + tests). L'objectif est de conserver la robustesse actuelle tout en gagnant progressivement en efficience.

2.1 Optimisations côté code produit (IMA)

2.1.1 Prologue/épilogue de méthodes plus sélectif

Actuellement, les méthodes sauvegardent/restaurent de manière conservatrice une plage de registres (`PUSH R2..Rmax` puis `POP`). Une piste d'amélioration est de ne sauvegarder que les registres effectivement utilisés (ou seulement les registres *caller-saved* selon une convention définie).
- **Gain attendu :** réduction du nombre d'instructions mémoire lors des appels de méthodes (donc baisse de cycles, notamment visible sur des programmes comme `ln2_fct`).
- **Risque/Coût :** nécessite une instrumentation (ex. suivi des registres alloués dans une méthode) ou une petite analyse de vivacité locale ; il faudra renforcer la validation.

2.1.2 Constant folding : réintroduction progressive et sûre

Nous avions tenté le constant folding puis l'avons retiré pour préserver la robustesse. Une piste raisonnable serait de le réintroduire **uniquement** sur un sous-ensemble sûr :
- littéraux entiers *sans overflow* (ou avec détection explicite) ;
- littéraux flottants en excluant `NaN`/`Inf` et en respectant les mêmes diagnostics ;
- sans modifier les chemins d'erreurs attendus (ex. cohérence avec `BOV`).

Gain attendu : réduction d'instructions sur boucles/initialisations, sans complexifier toutes les expressions.

2.1.3 Réduction des `LOAD` inutiles : instructions IMA à opérande mémoire

IMA permet certaines opérations arithmétiques directement avec un opérande mémoire (ex. `MUL addr, Ri`, `ADD addr, Ri`, etc.). Une optimisation simple consiste à **éviter un** `LOAD` quand l'un des opérandes est déjà en mémoire et que l'instruction accepte une adresse (`DAddr`) en opérande.

Exemple. Considérons :

```
{
  int x = 1;
```

```
  int y = 2;
  x = 3 * x - y / 2;
}
```

où `x` est à l'adresse `3(GB)` et `y` à l'adresse `4(GB)`. Le code produit peut être :

```
LOAD #3, R2          ; R2 := 3
MUL 3(GB), R2        ; R2 := 3 * x        (pas besoin de LOAD x)
LOAD 4(GB), R3       ; R3 := y
QUO #2, R3           ; R3 := y / 2
SUB R3, R2           ; R2 := R2 - R3
STORE R2, 3(GB)      ; x := résultat
```

Gain attendu. Par rapport à une stratégie plus naïve qui ferait `LOAD 3(GB), Ri` avant la multiplication, on économise au moins un accès mémoire (`LOAD`) sur le chemin chaud. Cela diminue :

- le nombre d'instructions,
- le nombre de cycles `ima -s`,
- et donc l'énergie consommée à l'exécution.

Où l'implémenter. Cette optimisation est typiquement locale dans les nœuds d'opérations arithmétiques (génération de `MUL/ADD/SUB/QUO`) : si un des fils est un `Identifier` (variable) et que l'autre est un registre.

2.2 Optimisations côté procédé de fabrication (Java/Maven/Tests)

2.2.1 Exécution parallèle des tests : à utiliser avec discernement

L'option parallèle (ex. `-P`) peut réduire le `real` (temps mur) mais augmenter le `user` (car plusieurs coeurs travaillent). Dans une perspective énergétique, on peut :

- garder le mode parallèle seulement pour des campagnes longues (stabilisation/CI) ;
- utiliser le mode séquentiel pour les itérations rapides de développement.

Gain attendu : meilleur compromis selon le contexte (rapidité vs consommation CPU totale).

Conclusion

Notre démarche vise une sobriété numérique **réaliste** dans le cadre du projet :

- Côté **code produit**, nous diminuons le nombre de cycles (approximation d'énergie sur `ima`) en privilégiant les registres, en contrôlant le spill, et en appliquant l'évaluation paresseuse pour les booléens.
- Côté **fabrication**, nous limitons l'impact des tests par une exécution ciblée et une organisation des scripts, tout en maintenant une bonne qualité de validation.
- Enfin, nous avons assumé des choix conservateurs (ex. sauvegarde large des registres en méthodes, abandon du constant folding) pour garantir la robustesse du compilateur et des messages d'erreurs : réduire les bugs réduit aussi le coût énergétique du développement (moins de cycles de debug/test).